

Android 第六章第四节教学讲义（Jetpack Room 实战版）

各位同学，今天咱们学习 Android 官方推荐的数据库框架——**Jetpack Room**。之前学的 SQLite 需要写大量重复代码，而 Room 就像给 SQLite 加了 "自动挡"，让数据库操作更简单、更安全。最后咱们还用 Room 实现一个完整的 "购物车" 案例，从数据库设计到功能实现一步到位，保证小白也能跟着做！

一、Jetpack Room：让数据库操作变简单

Room 是 Google 推出的 Jetpack 组件之一，它封装了 SQLite，不用再写复杂的 SQL 语句和游标操作，直接用注解和 Java 对象就能操作数据库，就像 "用面向对象的方式操作数据库"。

Room 的三个核心部分

- **Entity**：实体类，对应数据库中的一张表（用@Entity注解）；
- **Dao**：数据访问对象，定义增删改查的方法（用@Dao注解）；
- **Database**：数据库类，管理数据库版本和表结构（用@Database注解）。

三者关系：Database包含多个Entity（表），通过Dao操作Entity中的数据。

二、Room 基础用法：从 0 到 1 创建数据库

以 "用户表" 为例，演示 Room 的基本使用步骤。

步骤 1：添加 Room 依赖

在app/build.gradle中添加依赖（最新版本可查官网）：

```
dependencies {  
    // Room核心库  
    implementation "androidx.room:room-runtime:2.5.2"  
    // 注解处理器（编译时生成代码）  
    kapt "androidx.room:room-compiler:2.5.2"
```

```
// 支持在主线程操作数据库（仅测试用，正式环境用异步）
implementation "androidx.room:room-ktx:2.5.2"
}
```

点击 "Sync Now" 同步依赖。

步骤 2：创建 Entity（实体类 = 表结构）

用@Entity注解标记类，类中的字段对应表的列，@PrimaryKey标记主键。

```
// 用户表（user）
@Entity(tableName = "user") // tableName指定表名，默认是类名
public class User {
    @PrimaryKey(autoGenerate = true) // 主键自增长
    private int id;
    @ColumnInfo(name = "user_name") // 自定义列名，默认是字段名
    private String username;
    private String password;
    // 必须有默认构造方法（Room需要）
    public User() {}
    // 有参构造方法（方便创建对象）
    public User(String username, String password) {
        this.username = username;
        this.password = password;
    }
    // getter和setter方法（必须有，否则Room无法访问字段）
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

步骤 3：创建 Dao（数据访问接口）

用@Dao注解标记接口，定义增删改查方法，用注解代替 SQL 语句。

```

@Dao
public interface UserDao {
    // 增加：插入用户
    @Insert
    void insert(User user);
    // 删除：根据用户对象删除
    @Delete
    void delete(User user);
    // 修改：更新用户
    @Update
    void update(User user);
    // 查询：查询所有用户
    @Query("SELECT * FROM user")
    List<User> getAllUsers();
    // 查询：根据用户名查用户
    @Query("SELECT * FROM user WHERE user_name = :username")
    User getUserByUsername(String username);
}

```

注解说明：@Insert/@Delete/@Update会自动生成 SQL，@Query需要写 SQL 语句（:参数名表示传入的参数）。

步骤 4：创建 Database（数据库类）

用@Database注解标记，继承RoomDatabase，定义数据库版本和包含的表。

```

@Database(
    entities = {User.class}, // 包含的表（实体类）
    version = 1, // 数据库版本
    exportSchema = false // 不导出数据库 schema（简化用）
)
public abstract class AppDatabase extends RoomDatabase {
    // 单例模式（防止创建多个数据库实例）
    private static AppDatabase instance;
    public static synchronized AppDatabase getInstance(Context context) {
        if (instance == null) {
            // 创建数据库实例
            instance = Room.databaseBuilder(
                context.getApplicationContext(),
                AppDatabase.class,

```

```

        "app_database" // 数据库名
    ).allowMainThreadQueries() // 允许在主线程操作（仅测试用）
    .build();
}
return instance;
}
// 提供Dao的访问方法
public abstract UserDao userDao();
}

```

注意：正式开发中禁止`allowMainThreadQueries()`，需用异步操作（如`AsyncTask`或协程）。

步骤 5：使用 Room 操作数据库

```

// 获取数据库实例
AppDatabase db = AppDatabase.getInstance(this);
UserDao userDao = db.userDao();
// 1. 插入用户
User user = new User("张三", "123456");
userDao.insert(user);
// 2. 查询所有用户
List<User> userList = userDao.getAllUsers();
// 3. 根据用户名查询
User zhangsan = userDao.getUserByUsername("张三");
// 4. 更新用户（修改密码）
if (zhangsan != null) {
    zhangsan.setPassword("654321");
    userDao.update(zhangsan);
}
// 5. 删除用户
if (zhangsan != null) {
    userDao.delete(zhangsan);
}

```

三、实战：用 Room 实现购物车功能

咱们实现一个完整的购物车，包含 "商品列表展示"、"加入购物车"、"购物车列表"、"清空购物车" 等功能。

1. 数据库准备：创建商品表和购物车表

(1) 商品表 (Product)

```
@Entity(tableName = "product")
public class Product {
    @PrimaryKey
    private int id; // 商品ID (唯一)
    private String name; // 商品名称
    private String price; // 商品价格
    private String image; // 商品图片路径
    private String description; // 商品描述
    // 构造方法、getter、setter (略，需手动添加)
}
```

(2) 购物车表 (Cart)

```
@Entity(tableName = "cart")
public class Cart {
    @PrimaryKey(autoGenerate = true)
    private int id; // 购物车记录ID
    private int productId; // 关联的商品ID
    private int count; // 商品数量
    // 构造方法、getter、setter (略)
}
```

(3) 创建 Dao 接口

```
// 商品Dao
@Dao
public interface ProductDao {
    @Insert
    void insertAll(Product... products); // 批量插入商品
    @Query("SELECT * FROM product")
```

```

    List<Product> getAllProducts(); // 查询所有商品
    @Query("SELECT * FROM product WHERE id = :id")
    Product getProductById(int id); // 根据ID查商品
}
// 购物车Dao
@Dao
public interface CartDao {
    @Insert
    void addToCart(Cart cart); // 加入购物车
    @Query("SELECT * FROM cart")
    List<Cart> getAllCartItems(); // 查询购物车所有商品
    @Delete
    void removeFromCart(Cart cart); // 从购物车移除
    @Query("DELETE FROM cart")
    void clearCart(); // 清空购物车
    @Update
    void updateCartItem(Cart cart); // 更新购物车商品数量
}

```

(4) 更新数据库类

```

@Database(
    entities = {Product.class, Cart.class}, // 包含两个表
    version = 1
)
public abstract class AppDatabase extends RoomDatabase {
    // 单例代码同上 (略)
    public abstract ProductDao productDao();
    public abstract CartDao cartDao();
}

```

2. 初始化商品数据（模拟服务器加载）

在 APP 启动时往商品表插入几条测试数据：

```

// 在Application的onCreate()中初始化
public class MyApplication extends Application {
    @Override

```

```

public void onCreate() {
    super.onCreate();
    // 子线程初始化商品数据
    new Thread(() -> {
        AppDatabase db = AppDatabase.getInstance(this);
        ProductDao productDao = db.productDao();
        // 如果表中没有数据，插入测试商品
        if (productDao.getAllProducts().isEmpty()) {
            productDao.insertAll(
                new Product(1, "小米手机", "3999", "phone1", "高性能手机"),
                new Product(2, "华为平板", "2999", "pad1", "轻薄平板"),
                new Product(3, "苹果耳机", "1299", "ear1", "无线耳机")
            );
        }
    }).start();
}
}

```

3. 商品列表展示

用 RecyclerView 展示所有商品：

(1) 布局 (activity_product_list.xml)

```

<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/rv_products"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>

```

(2) 代码逻辑

```

public class ProductListActivity extends AppCompatActivity {
    private RecyclerView rvProducts;
    private ProductAdapter adapter;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_product_list);
    }
}

```

```

rvProducts = findViewById(R.id.rv_products);
rvProducts.setLayoutManager(new LinearLayoutManager(this));
adapter = new ProductAdapter();
rvProducts.setAdapter(adapter);
// 加载商品数据
loadProducts();
}
private void loadProducts() {
    new Thread(() -> {
        // 从数据库查询所有商品
        List<Product> products = AppDatabase.getInstance(this).productDao().getAllProducts();
        // 主线程更新UI
        runOnUiThread(() -> adapter.setData(products));
    }).start();
}
// 商品适配器（略，绑定商品名称、价格，添加"加入购物车"按钮）
}

```

4. 加入购物车功能

在商品列表的 "加入购物车" 按钮点击事件中实现：

```

// 适配器中按钮点击事件
holder.btnAddToCart.setOnClickListener(v -> {
    Product product = products.get(holder.getAdapterPosition());
    new Thread(() -> {
        CartDao cartDao = AppDatabase.getInstance(context).cartDao();
        // 先查询购物车中是否已有该商品
        List<Cart> cartItems = cartDao.getAllCartItems();
        Cart existingCart = null;
        for (Cart item : cartItems) {
            if (item.getProductId() == product.getId()) {
                existingCart = item;
                break;
            }
        }
    })
    if (existingCart != null) {
        // 已有则数量+1
        existingCart.setCount(existingCart.getCount() + 1);
    }
}

```



```

        cartDao.updateCartItem(existingCart);
    } else {
        // 没有则新增购物车记录（数量=1）
        cartDao.addToCart(new Cart(product.getId(), 1));
    }
    // 主线程提示
    ((Activity) context).runOnUiThread() ->
        Toast.makeText(context, "已加入购物车", Toast.LENGTH_SHORT).show()
    );
    }.start();
});

```

5. 购物车列表展示

类似商品列表，查询 Cart 表，再关联 Product 表获取商品详情：

```

private void loadCartItems() {
    new Thread() -> {
        CartDao cartDao = AppDatabase.getInstance(this).cartDao();
        ProductDao productDao = AppDatabase.getInstance(this).productDao();
        List<Cart> cartItems = cartDao.getAllCartItems();
        // 关联商品信息（购物车记录+商品详情）
        List<CartItem> cartItemList = new ArrayList<>();
        for (Cart cart : cartItems) {
            Product product = productDao.getProductById(cart.getProductId());
            cartItemList.add(new CartItem(cart, product));
        }
        // 主线程更新UI
        runOnUiThread() -> cartAdapter.setData(cartItemList);
    }.start();
}

```

6. 清空购物车功能

```

// 清空购物车按钮点击事件
btnClearCart.setOnClickListener(v -> {
    new Thread() -> {

```

```
AppDatabase.getInstance(this).cartDao().clearCart();
// 主线程刷新列表
runOnUiThread(() -> {
    cartAdapter.clear();
    Toast.makeText(this, "购物车已清空", Toast.LENGTH_SHORT).show();
});
}).start();
});
```

7. 商品详情页

点击商品列表项跳转到详情页，根据商品 ID 查询详情：

```
// 商品列表点击事件
holder.itemView.setOnClickListener(v -> {
    Product product = products.get(holder.getAdapterPosition());
    Intent intent = new Intent(context, ProductDetailActivity.class);
    intent.putExtra("product_id", product.getId());
    context.startActivity(intent);
});
// 详情页获取数据
int productId = getIntent().getIntExtra("product_id", -1);
new Thread(() -> {
    Product product = AppDatabase.getInstance(this).productDao().getProductById(productId)
;
    runOnUiThread(() -> {
        // 显示商品名称、价格、描述等
        tvName.setText(product.getName());
        tvPrice.setText(product.getPrice());
    });
}).start();
```

总结

这节课咱们学了 Jetpack Room 的核心用法和购物车实战：

1. Room 三要素：

- Entity：用注解定义表结构（类 = 表，字段 = 列）；

- Dao：用注解定义增删改查方法（@Insert/@Query等）；
- Database：管理数据库版本和表，通过单例获取实例。

2. 购物车实现步骤：

- 设计商品表（Product）和购物车表（Cart）；
- 用ProductDao和CartDao实现数据操作；
- 依次实现商品列表、加入购物车、购物车列表、清空购物车等功能。

Room 是现在 Android 开发的主流数据库框架，相比原生 SQLite 更简洁、更不易出错。课后可以给购物车添加 "修改商品数量" 和 "删除单个商品" 功能，巩固今天的知识。有问题随时提问哦！

（注：文档部分内容可能由 AI 生成）